

SAPIENZA
UNIVERSITÀ DI ROMA

Secure Satellite Communication: Performance and cryptographic analysis of chaotic injected AES resistant to side-channel attacks targeting the S-box via machine unlearning

Faculty of Information Engineering, Computer Science and Statistics
Bachelor's Degree in Computer Science

Simone Descontus
ID number 2116496

Advisor
Prof. Emiliano Casalicchio

Academic Year 2025/2026

Secure Satellite Communication: Performance and cryptographic analysis of chaotic injected AES resistant to side-channel attacks targeting the S-box via machine unlearning.
Sapienza University of Rome

© 2026 Simone Descontus. All rights reserved.

This thesis has been typeset by LaTeX.

Author's email: simo.desk04@gmail.com

Contents

1 Introduction **3**
1.1 Thesis Objective . . . 3

2 Background and State-of-the-Art of Satellite Communications **4**
2.1 SATCOM Architecture . . . 5
2.1.1 Space Segment . . . 6
2.1.2 Ground Segment . . . 6
2.1.3 Link Segment . . . 7
2.2 SATCOM Threats . . . 7
2.2.1 Space Segment Threats . . . 7
2.2.2 Ground Segment Threats . . . 8
2.2.3 Link Segment Threats . . . 8
2.2.4 An Emergent Threat: Machine Learning . . . 9
2.3 Defensive Strategies . . . 10
2.3.1 Physical Layer Security (PLS) . . . 10
2.3.2 Traditional Cryptography . . . 10
2.4 SATCOM Cryptographic State-of-the-Art: AES-256-GCM . . . 11

3 Algorithmic Foundations of AES-256-GCM **13**
3.1 AES-256 . . . 13
3.1.1 Key Expansion . . . 14
3.1.2 State Matrix and Round Architecture . . . 15
3.1.3 SubBytes . . . 16
3.1.4 ShiftRows . . . 17
3.1.5 MixColumns . . . 18
3.1.6 AddRoundKey . . . 20
3.2 Galois/Counter Mode (GCM) . . . 20
3.2.1 Key and IV Requirements . . . 22
3.2.2 Multiplication in GF(2^{128}) . . . 23
3.2.3 GHASH Function . . . 24
3.2.4 Authenticated Encryption and Decryption Algorithms . . . 26
3.2.5 Tests for the GCM Embedded C Implementation . . . 28

4 Chaotic Injection and Cryptographic Machine Unlearning **30**

Bibliography **31**

Chapter 1

Introduction

1.1 Thesis Objective

Inspired by the foundational work of Kumar et al. [1], the primary objective of this thesis is to develop a proof-of-concept implementation of *AES with chaotic injection*. This is achieved by extending the open-source `tiny-aes-c` library [2] and adapting it to comply with modern satellite communication standards through the integration of the *AES-256-GCM* mode of operation [3].

Furthermore, this study aims to analyze the integration of various chaotic maps within the cipher's architecture. These configurations will be rigorously benchmarked to evaluate execution time and energy consumption, thereby identifying the optimal solution for the resource-constrained SATCOM environment. Concurrently, statistical and cryptographic tests will verify that altering the standard encryption algorithm does not degrade its fundamental cryptographic properties.

Building upon these analyses, the thesis proposes an architectural optimization tailored for satellite communications. A comprehensive comparison will be provided between the baseline chaotic algorithm proposed by Kumar et al. and a modified iteration engineered to yield superior performance metrics, rendering it highly appropriate for space applications.

Lastly, significantly diverging from the scope of Kumar et al. [1], this research will simulate a Machine Learning-based Side-Channel Attack (SCA) aiming for complete key recovery. This simulation serves a critical dual purpose: it demonstrates a devastating threat vector that is entirely feasible against standard AES-256-GCM in current SATCOM architectures, while simultaneously proving that the proposed chaotic implementation successfully immunizes the system against such advanced profiling attacks.

Chapter 2

Background and State-of-the-Art of Satellite Communications

Satellite communications (SATCOM) are vastly different from how they appeared just a few decades ago. At that time, space-based communications were mainly represented by government activities, characterized by larger and more expensive satellites than the ones used today. This era is also known as the *old space* [4, p. 377], a term often used to describe the space applications that emerged during the Cold War.

On the other hand, since the beginning of the *new space* era [4, p. 377–398], thanks to the ever-decreasing cost of satellite launches and the enormous technological progress that has been made in recent years, space has become increasingly populated by satellites used for commercial purposes, such as:

- *Telecommunications and broadband connectivity*: they help to provide an internet connection even in the most remote regions or those that are hard to reach with a physical connection [4, p. 373–381].
- *Television broadcast and multimedia streaming*: due to the constant demand for high-quality multimedia, the wide coverage of satellite communications helps to deliver content to a large number of users more efficiently [4, p. 373–381].
- *Global Positioning System (GPS)*: numerous sectors make use of satellite-based geolocation to plan routes or monitor the movement of resources. In addition to this, a large number of people use geolocation daily to plan their travels [4, p. 378–381].
- *Earth observation*: thanks to more affordable satellites, it is now possible for private entities to start their own Earth observation programs for scientific purposes, such as environmental analysis. Moreover, satellites observing the Earth are widely used for meteorological purposes [4, p. 372–381].
- *Internet of Things (IoT)*: because of the multifaceted nature of IoT devices, which can take on many different forms, from smart homes to means of transport, it is essential that the coverage area serving these devices is as wide as possible. Such needs are effectively addressed by using SATCOMs as the main communication method [4, p. 372].
- *Satellite as a service (SataaS)*: while it is true that the cost of satellite lounches is significantly decreased, it remains unaffordable for many private companies that would otherwise

benefit of the wide coverage provided by SATCOMs.

To address this challange, majour caerospace operators can lease resources on their satellites to third-party users, granting them access to space infrastructure without the need for additional launches [5].

Along with the opportunities offered by the *new space*, the critical requirements of the *old space* have not diminished. Today, satellite communications remain essential for handling mission-critical data exchanges, such as those required by aviation and maritime operations [4, p. 397–398], as well as for the coordination of rescue services in disaster or crisis situations [4, p. 388–389].

Given the critical nature of the communications handled by SATCOM systems, it is becoming increasingly important to protect the information shared within these networks from malicious actors. To fully understand the scale of these security challenges and how threats can compromise the network, it is essential to analyze its underlying infrastructure. A modern SATCOM system is not a monolithic entity, but rather a complex, distributed architecture. Breaking down this infrastructure into its fundamental segments is a crucial step to accurately identify the specific attack surfaces that adversaries might exploit.

2.1 SATCOM Architecture

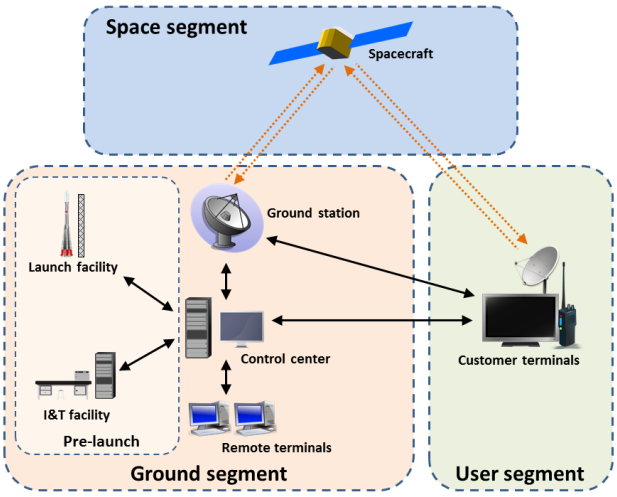


Figure 2.1:
Architectural overview of space, ground, and user segments in satellite systems [6].

Satellite communications are typically classified into three fundamental segments, although the exact categorization may vary across the literature. One common classification framework, as described in extended SATCOM state-of-the-art analysis provided by Salim et al. [4], divides the architecture into the *space segment*, the *ground segment*, and the *link segment*. In this model, the *ground Segment* encompasses both the *ground Stations*, which act as gateways for devices lacking direct satellite communication capabilities, and the *user segment*, which consists of end-user terminals equipped for space links. Another classification model still includes the *space segment* and

the *ground segment*, but explicitly separates the *user segment*, treating the *link segment* merely as a passive communication channel [7, p. 5671].

While all these components are crucial to maintain the network's operability, this thesis will adopt the classification proposed in [4]. Grouping the ground stations and end-user devices under a unified *ground segment* is conceptually sound for terrestrial entities. More importantly, treating the *link segment* as a primary, distinct architectural component is essential for the scope of this study, as the physical transmission channel introduces specific vulnerabilities and security challenges that are central to the analyzed threat model.

2.1.1 Space Segment

The space segment is represented by the satellites in orbit, each composed of the *bus*, which handles onboard systems such as power and thermal management, and the *payload*, which is the part of the spacecraft designed to fulfill the satellite's specific mission [4, p. 381–383]. Examples of payloads include telescopes to analyze distant objects or sensors to measure spatial properties.

Depending on the satellite's mission, its orbital height from Earth can vary radically [7]:

- *Low Earth Orbit (LEO)*: due to their low altitude, usually between 500 and 1200 km, these types of satellites are ideal for extending terrestrial networks thanks to their low latency (around 30 ms). However, because they are so close to the Earth, it is often necessary to deploy a group of interconnected *LEO* satellites, known as a *satellite constellation*, in order to cover a wider area of the planet. Additionally, because of the high risk of collision with space debris, the lifetime of *LEO* spacecraft is around 5 to 7 years, which is usually shorter than that of other satellites.

- *Medium Earth Orbit (MEO)*: usually positioned at an altitude of 2000 to 20000 km, they represent a midpoint between coverage and latency (around 125 ms). A significant share of *MEO* spacecraft function as GPS satellites, and typically 6 units are necessary in order to provide adequate service. Because of their intermediate position, *MEO* satellites often serve as relay nodes between *LEO* and *GEO* satellites. Finally, their operational lifetime is estimated at 10 to 15 years, depending on their altitude and exposure to radiation.

- *Geostationary Equatorial Orbit (GEO)*: operating in the highest orbital layer at approximately 36000 km, these spacecraft take exactly 24 hours to complete an orbit, appearing stationary to an observer on Earth. Thanks to their high altitude, they can cover one-third of the entire planet's surface, making them ideal for broadcasting. However, it is important to account for the latency accumulated during transmission (around 600 ms). Due to their distance from the planet and the large quantity of resources needed to launch *GEO* spacecraft, they are usually built to last longer, typically around 15 to 20 years.

2.1.2 Ground Segment

This segment represents the terrestrial infrastructure that communicates with satellites. Depending on the role of the receiver or sender, it can be classified into several sub-components, the primary ones being [4, p. 383–384]:

- *Ground Stations (GS)*: equipped with dedicated hardware, they can transmit and receive signals even from the most distant satellites. A vast amount of traffic passes through them, as the majority of terrestrial devices lack the necessary peripherals to communicate directly with spacecraft.
- *Mission Control Centers (MCC)*: their role is to ensure the satellite's mission continues undisturbed by analyzing the telemetry sent by the spacecraft. Telemetry consists of messages transmitted by the satellites containing critical information, such as their exact orbit or internal hardware measurements; this data allows operators to detect anomalies and perform necessary corrective actions.
- *User Segment*: this component comprises all the end-user devices, which receive messages after they are forwarded by the *Ground Stations*. Moreover, devices equipped with the necessary peripherals to communicate directly with the *space segment*, such as ships, airplanes, and Satellite TV, also fall into this category.

2.1.3 Link Segment

This segment is represented by the radio waves that link the Earth to space and the satellites to one another, as well as the hardware components used to establish these connections. Specifically, the *link segment* can be categorized as follows [4, p. 381]:

- *Ground-to-Ground*: connects the *ground stations* to the *user segment* and vice versa.
- *Ground-to-Space*: also known as the *uplink*, it represents the communication path from the *ground segment* to the *space segment*.
- *Space-to-Ground*: also known as the *downlink*, it represents the communication path from the *space segment* to the *ground segment*.
- *Space-to-Space*: connects the spacecraft within the *space segment* to one another, even across different orbital altitudes.

2.2 SATCOM Threats

Due to the distributed architecture of SATCOM networks, security threats targeting any individual segment must be rigorously addressed. Tampering with a single component can have cascading effects, jeopardizing the other segments and compromising both the spacecraft's mission and the broader network to which it is connected. Some threats associated with each segment are outlined below.

2.2.1 Space Segment Threats

While spacecraft are physically out of reach for potential attackers, this physical isolation does not exempt the *space segment* from cyber threats.

Examples include *software attacks*, in which malicious software could be introduced into the satellite's systems via the *link segment*. Alternatively, in a *Satellite-as-a-Service (SataaS)* architecture, an attacker could exploit the shared infrastructure by gaining access to the spacecraft's resources under the guise of a legitimate user. Furthermore, *Denial of Service (DoS)* attacks pose a significant threat to the *space segment*, as an excessive volume of requests sent to the targeted satellite could overwhelm its processing capabilities, temporarily or permanently disrupting its mission.

Beyond software vulnerabilities, *hardware attacks* target the physical infrastructure of the satellite. A particularly insidious example is the *supply chain attack*, which compromises the manufacturing process of the spacecraft by exploiting the decentralized production models typical of the *new space* industry [4, p. 386–389].

Although significantly harder to mount and execute, another critical threat to the *space segment* is posed by *kinetic or anti-satellite (ASAT) weapons*. These systems consist of high-energy beams or physical interceptors that aim to blind the spacecraft's communications or physically destroy its components [7, p. 5677].

Finally, another catastrophic threat is the risk of *satellite hijacking*. Through command spoofing techniques, an attacker could force the spacecraft to deviate from its intended orbit, often leading to the complete destruction of the satellite as it burns up upon atmospheric reentry [4, p. 386–389].

2.2.2 Ground Segment Threats

Compared to the *space segment*, the *ground segment* represents a much more accessible entry point for attackers due to its terrestrial location. Moreover, despite the physical distance between *Ground Stations* and the spacecraft, the impairment of a terrestrial facility can easily jeopardize the satellite itself. For instance, malware contaminating a station on Earth could easily be transferred to the *space segment* via the *link segment*.

Similar to the threats facing the space infrastructure, *software attacks* could lead to the temporary deactivation of the communication channels used to send messages to the *space segment*. If the compromised facility is a *Mission Control Center (MCC)*, operators would be unable to monitor the satellite's telemetry or detect potential anomalies. Consequently, a synchronized cyberattack targeting the *Space Segment* simultaneously would go completely unnoticed.

While *Denial of Service (DoS)* and *Distributed Denial of Service (DDoS)* attacks also pose a significant threat to the *ground segment*, a vulnerability exclusively tied to the terrestrial infrastructure is *Unauthorized Access to Ground Stations*. If physical or logical authentication mechanisms are missing, poorly designed, or vulnerable, an adversary could gain unauthorized entry, ultimately leading to a complete takeover of the spacecraft's control systems [4, p. 389–391].

2.2.3 Link Segment Threats

Lastly, the *link aegment* is arguably the most vulnerable of the three segments, primarily due to its wireless nature. The most straightforward threat is *eavesdropping*, which consists of the unauthorized interception of communications. The adversaries' preferred target for eavesdropping

is typically the *downlink*, as the signal is directed towards Earth over a wide footprint; conversely, the highly directional trajectory of the *uplink* makes it significantly more difficult to capture data [4, p. 393].

Link jamming is another critical threat to consider. Equipped with sufficiently powerful hardware, adversaries can execute a *DoS* attack by flooding the communication channel with noise. Just as with eavesdropping, it is generally easier to jam the *downlink* than the *uplink*. This occurs because the signal arriving from space is inherently weak by the time it reaches Earth, whereas a terrestrial attacker would find it extremely challenging to overpower the highly amplified *uplink* signals transmitted by *Ground Stations*.

Furthermore, *signal spoofing* involves falsifying or replaying messages over the communication link. By doing so, an attacker can deceive the receiving system if robust security and authentication mechanisms are missing or insufficient [4, p. 391–394].

2.2.4 An Emergent Threat: Machine Learning

Although *Artificial Intelligence (AI)* and *Machine Learning (ML)* are widely employed in defensive contexts, such as *Intrusion Detection Systems (IDS)* and *Intrusion Prevention Systems (IPS)*, they also represent a formidable offensive threat.

A first example is linked to the energy-constrained nature of the space environment. Due to strict limits on power and computational resources, onboard ML models dedicated to security are often lightweight and less complex. Consequently, attackers can exploit these limitations by injecting imperceptible adversarial noise to elude the defensive models [7, p. 5682].

More critically for the scope of this study, the increasing accessibility of advanced ML algorithms has revolutionized cryptanalysis. While ML can be used to perform traffic analysis by identifying patterns in encrypted SATCOM communications due to the presence of predictable packet headers, an even more critical risk arises when ML is combined with *Side-Channel Attacks (SCA)*.

Adversaries can leverage ML models to analyze physical leakages, such as the power consumption of the satellite's hardware during encryption. Thanks to the ability of ML to uncover complex, non-linear correlations within noisy data, attackers can profile these leakages and completely recover the secret keys. In the context of shared architectures like *SataaS*, where adversaries might run software on the same physical hardware, ML-based SCAs represent a severe threat if the isolation mechanisms designed to prevent software-based power monitoring are missing or insufficient.

Furthermore, while the traditional execution of SCAs requires placing specific measurement probes on the target hardware, which is an unfeasible task for a satellite already in orbit, a sophisticated *supply chain attack* can circumvent this limitation. Adversaries could compromise key satellite components, such as cryptographic chips or CPUs, by implanting the necessary hardware trojans during the manufacturing process, establishing a covert channel to transmit the leaked information once deployed in space.

Lastly, to compromise the SATCOM infrastructure via an SCA, an adversary could exploit the interconnection between the network segments by targeting the more physically accessible *ground*

stations. Since the ground and space segments must share the same cryptographic parameters to establish a secure link, successfully extracting the secret key from a ground terminal yields the exact same catastrophic result for the entire network.

2.3 Defensive Strategies

To counter the vast range of threats discussed previously, SATCOM security relies on various defensive strategies that can be broadly categorized into two main paradigms. While both approaches aim to secure satellite communications, they achieve this through fundamentally different methodologies.

2.3.1 Physical Layer Security (PLS)

The first approach leverages the intrinsic physical properties of the wireless communication channel to provide countermeasures against specific threats. It minimizes resource consumption while ensuring a high level of security under targeted operational circumstances.

A prominent example of this technique relies on the concept of the *wiretap channel*. By employing specialized beamforming antennas, the sender can focus the transmission beam, confining the signal's footprint exclusively to the legitimate receiver's location. Simultaneously, artificial noise can be injected into the spatial directions outside this main beam to degrade the signal-to-noise ratio for potential eavesdroppers [8, p. 3–4].

A major advantage of *PLS* is that it is significantly less demanding in terms of energy consumption compared to high-level *cryptography* [8, p. 2]. Consequently, this technology is particularly well-suited for SATCOM environments, where spacecraft must operate under strict energy and computational constraints.

However, this defensive strategy presents notable limitations. For instance, PLS mechanisms cannot detect or correct data integrity issues, such as bit-flip errors, which frequently occur during transmission due to long propagation distances and the interference of space radiation. To address these vulnerabilities, current literature frequently explores the combined use of *PLS* and high-level *cryptography*. This hybrid approach aims to achieve comprehensive security by allowing the two methodologies to mutually compensate for each other's limitations.

2.3.2 Traditional Cryptography

Regardless of the transmission medium, *cryptography* represents a fundamental solution for ensuring data confidentiality. By relying on either a single secret key or a public-private key pair, cryptographic algorithms conceal the true content of a message, rendering the transmitted data indistinguishable from random noise.

Algorithms that rely on a single shared key are known as symmetric-key ciphers. For this cryptographic approach to function securely, the secret key must be distributed through a secure channel; the compromise of such critical information leads to a complete breach of communication confidentiality until a new secret key can be established.

To address the key distribution challenge, researchers continuously explore lightweight and efficient methods to establish secure channels. Proposed solutions include *Identity-Based Encryption (IBE)*, where the receiver's public identity serves as the public key while a trusted third party generates the corresponding private key [9, pp. 2193–2194], as well as *Quantum Key Distribution (QKD)*, which leverages quantum mechanics to securely establish a symmetric key [7, p. 5683]. Traditionally, however, SATCOM systems have relied on pre-loading a set of master keys onto the spacecraft prior to its launch, from which dynamically changing session keys are derived [10]. More recently, modern space infrastructures have begun deploying *Elliptic Curve Cryptography (ECC)-based Diffie-Hellman* key exchanges, capitalizing on the computational efficiency and smaller key sizes of ECC [11].

As implied by the adoption of ECC, the resource-constrained nature of the space environment implies the use of highly efficient algorithms to avoid overburdening the satellite's power and computational budget. For this reason, traditional asymmetric-key algorithms like *RSA*, which require substantial computational overhead to process large key pairs, are generally not recommended for SATCOM encryption.

Finally, a major advantage of traditional cryptography over PLS techniques lies in its software-based nature. While deploying a new PLS mechanism requires rigid hardware integration during the satellite's manufacturing phase, cryptographic protocols can potentially be updated or patched remotely, even for spacecraft already in active mission operations.

Furthermore, to combat both eavesdropping and signal spoofing simultaneously, modern SATCOM architectures increasingly rely on Authenticated Encryption with Associated Data (AEAD) schemes, which guarantee both the confidentiality and the integrity of the transmitted information in a single, efficient operational step [3].

2.4 SATCOM Cryptographic State-of-the-Art: AES-256-GCM

Following the directives of the *Consultative Committee for Space Data Systems (CCSDS)*, the most widely adopted cryptographic algorithm in modern satellite infrastructures is the *Advanced Encryption Standard (AES)* operating in *Galois/Counter Mode (GCM)* with a 256-bit key [3].

The rationale behind this choice rests on the intrinsic computational efficiency of AES. While older deployments often relied on 128-bit keys, the escalating threat posed by quantum computing has shifted the security paradigm. Quantum-based attacks, specifically those leveraging Grover's algorithm, can effectively halve the search space of a symmetric-key block cipher, reducing a 128-bit key security level from 2^{128} to a computationally feasible 2^{64} [12]. Conversely, halving the 2^{256} search space of AES-256 still leaves an effective security level of 2^{128}, which remains robustly secure against foreseeable quantum cryptanalysis.

AES is a block cipher that requires its input to be processed in fixed blocks of 128 bits. To adapt this core algorithm to variable-length messages, a specific *mode of operation* must be employed. The CCSDS recommends GCM, an advanced alteration of the classic *Counter Mode (CTR)*. GCM acts as an *Authenticated Encryption with Associated Data (AEAD)* scheme, providing both message confidentiality and authentication simultaneously. This is achieved by generating

an authentication tag derived from the plaintext, the secret key, and the *Additional Authenticated Data (AAD)*.

The adoption of GCM ensures that any data manipulation or bit-flip errors incurred during transmission can be reliably intercepted; if the authentication tag verification fails, the receiver can discard the corrupted payload and request a retransmission.

Finally, to guarantee cryptographic security, AES-GCM requires a strictly unique *Initialization Vector (IV)* associated with each message. Reusing an IV compromises the algorithm's confidentiality. Based on CCSDS recommendations, the optimal IV length is fixed at 96 bits (12 bytes) [12].

Chapter 3

Algorithmic Foundations of AES-256-GCM

As mentioned in the previous chapter, the primary cryptographic standard utilized in modern SATCOM environments is AES-256-GCM.

This chapter provides a comprehensive overview of the mathematical and algorithmic logic underlying this protocol. First, it explores the internal mechanics of the Advanced Encryption Standard (AES) block cipher, strictly following the specifications outlined in the *Federal Information Processing Standard (FIPS) 197* [13]. Subsequently, it details the *Galois/Counter Mode (GCM)* of operation, based on the guidelines established by the *National Institute of Standards and Technology (NIST)* Special Publication 800-38D [14].

By analyzing these standardized primitives, this chapter lays the essential technical groundwork for understanding the specific vulnerabilities targeted by Side-Channel Attacks (SCA), and ultimately justifies the architectural modifications proposed later in this study.

3.1 AES-256

The Advanced Encryption Standard (AES) has established itself as the preeminent symmetric encryption algorithm since the publication of the official *FIPS 197* standard in 2001. Selected by NIST through a rigorous public competition to replace the aging *Data Encryption Standard (DES)*, AES has consistently proven its resilience over the decades. The core algorithm has remained fundamentally unchanged, with the primary adaptation to emerging threats being the transition from a 128-bit to a 256-bit key length.

As a symmetric-key cipher, AES relies on a single shared secret key for both the encryption and decryption processes. This symmetry, combined with the algorithm's computational efficiency, results in a highly lightweight protocol that maintains robust cryptographic security. These characteristics represent the primary reasons for its widespread adoption as the standard for resource-constrained environments like SATCOMs.

To translate these theoretical advantages into a practical experimental framework, this study adopts the open-source `tiny-aes-c` library [2] as the baseline implementation. This C-based library provides a solid foundation for both experimental analysis and proof-of-concept alterations of the standard protocol. Its reliability and minimal computational overhead are well documented, as evidenced by its widespread use as a benchmark reference in recent literature concerning resource-constrained networks, such as Internet of Medical Things (IoMT) [15] and broader IoT

applications [16].

Returning to the theoretical framework, before the actual encryption process begins, the secret key must pass through the KEYEXPANSION routine. This function extends the original 32-byte key into a 240-byte key schedule, which is essential to provide a unique 128-bit round key for the initial key addition phase and for each of the subsequent 14 rounds of the algorithm.

3.1.1 Key Expansion

To comprehend the logic underlying the Key Expansion algorithm, the primary variables and constants are defined as follows:

- N_k: The number of 32-bit *words* comprising the original cipher key. For AES-256, $N_k = 8$.
- N_r: The number of rounds the algorithm will execute. For AES-256, it is fixed at $N_r = 14$.
- w: The linear array of 32-bit words that will hold the complete expanded key schedule.
- `temp`: A temporary 32-bit variable used during transformations.
- `Rcon`: A predefined array of round constant words, used to eliminate symmetries.

The procedure is detailed in the following pseudocode (Algorithm 1):

Algorithm 1 Key Expansion Algorithm for AES-256

```
1:  procedure KEYEXPANSION(key)
2:      i ← 0
3:      while i ≤ N_k − 1 do
4:          w[i] ← key[4·i, ..., 4·i+3]
5:          i ← i+1
6:      end while                          ▷ When the loop concludes, i = N_k.
7:      while i ≤ (4·N_r + 3) do
8:          temp ← w[i−1]
9:          if i mod N_k = 0 then
10:             temp ← SUBWORD(ROTWORD(temp)) ⊕ Rcon[i/N_k]
11:         else if N_k > 6 and i mod N_k = 4 then
12:             temp ← SUBWORD(temp)
13:         end if
14:         w[i] ← w[i−N_k] ⊕ temp
15:         i ← i+1
16:     end while
17:     return w
18: end procedure
```

In the initial phase (`lines 3-6`), the original secret key is sequentially copied into the first N_k words of the expanded key array w. In the specific case of AES-256 ($N_k = 8$), the array elements $w[0]$ through $w[7]$ will hold the unmodified original key.

The main loop (`lines 7-17`) iterates until the key schedule is completely populated, generating a total of $4 \cdot (N_r + 1)$ words (i.e., 60 words for AES-256). For each iteration, the word

preceding the current index is copied into the `temp` variable (`line 8`). Depending on the index i, specific non-linear operations may be applied to `temp` to guarantee cryptographic diffusion and security.

Specifically, whenever the index i marks the beginning of a new N_k-word block (i.e., i mod $N_k = 0$), `temp` undergoes a critical three-step transformation (`line 10`). First, RotWord applies a cyclic left permutation to the four bytes within the word, shifting `[a,b,c,d]` to `[b,c,d,a]`. Secondly, the SubWord function substitutes each individual byte using the non-linear AES S-box (which will be explored comprehensively later in this study). Lastly, the result is XORed with the round constant `Rcon` to disrupt any mathematical symmetries.

Furthermore, AES-256 introduces an additional security measure: when $N_k > 6$ (i.e., only for AES-256) and i mod $N_k = 4$, a secondary SubWord transformation is applied to the `temp` variable (`line 12`). This specific step was designed by the algorithm's creators to break potential patterns that could emerge due to the extended 256-bit key length.

Finally, the new word $w[i]$ is generated by computing a bitwise XOR between the heavily transformed `temp` variable and the expanded key word located exactly N_k positions prior (`line 14`).

3.1.2 State Matrix and Round Architecture

AES operates as a block cipher, meaning it processes data in fixed-size blocks of 128 bits (16 bytes). An input block can be represented as an array of 16 bytes:

$$\mathbf{in} = \begin{bmatrix} in_0 & in_1 & in_2 & \dots & in_{15} \end{bmatrix}^T$$

During the execution of the algorithm, this 16-byte input is mapped into a $4 \cdot 4$ matrix of bytes, known as the *State matrix* (S). As defined by the standard, the State matrix is populated column by column:

$$S = \begin{bmatrix} in_0 & in_4 & in_8 & in_{12} \\ in_1 & in_5 & in_9 & in_{13} \\ in_2 & in_6 & in_{10} & in_{14} \\ in_3 & in_7 & in_{11} & in_{15} \end{bmatrix}$$

As a testament to the structural elegance of the algorithm, AES-256 consists of 14 iterations (rounds) of four fundamental transformations applied to the State matrix in a fixed sequence: SubBytes, ShiftRows, MixColumns, and AddRoundKey. The only deviations from this strict regularity are an initial AddRoundKey transformation applied prior to the first round, and the deliberate omission of the MixColumns operation in the final (14th) round.

Finally, each AES transformation, with the exception of AddRoundKey, which is its own mathematical inverse (being a simple XOR operation), possesses a strictly corresponding inverse function utilized during the decryption phase. The decryption algorithm essentially reverses the encryption process by applying these inverse operations (e.g., InvSubBytes, InvShiftRows) in the reverse sequence. Consequently, the 128-bit round keys derived during the Key Expansion phase are also applied in reverse order, starting with the final round key and concluding with the

original secret key.

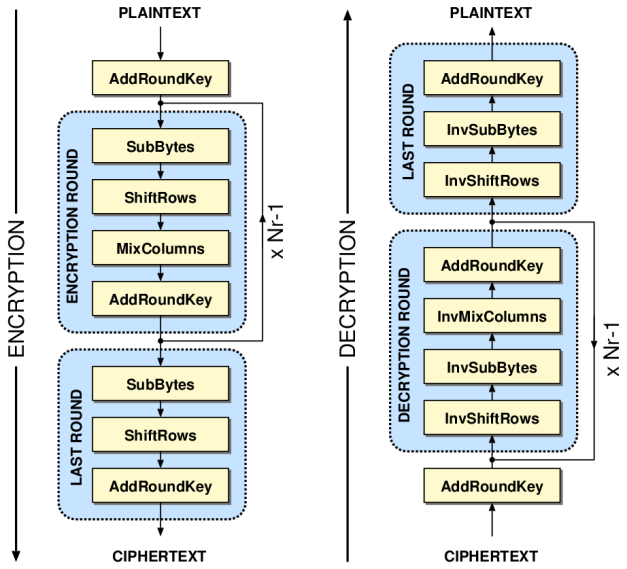


Figure 3.1: Internal structure of the AES algorithm [17].

3.1.3 SubBytes

The SUBBYTES operation is the only strictly non-linear transformation within the AES algorithm. It consists of a byte-by-byte substitution based on a fixed Look-Up Table (LUT) called the *S-Box*. The S-Box is a $16 \cdot 16$ matrix containing every possible byte value, ranging from `0x00` to `0xFF`.

Both the columns and the rows are labeled in hexadecimal format from `0` to `F`. To apply the substitution, each byte of the State matrix is treated as a two-digit hexadecimal value: the first digit determines the row, and the second digit determines the column. The substituted byte is the value located at these intersecting coordinates.

		y															
		0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
x	0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
	1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
	2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
	3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
	4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
	5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
	6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
	7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
	8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
	9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
	a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
	b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
	c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
	d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
	e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
	f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Figure 3.2: The AES Substitution Box (S-Box) [18].

For instance, considering the binary input $X = 01101100_2$, its hexadecimal representation is `0x6C`. By intersecting row `6` and column `C` in the S-Box, the output is `0x50`. Therefore, SUBBYTES(`0x6C`) = `0x50`.

The Construction of the S-Box The design of the S-Box stems from the fundamental cryptographic necessity to introduce strong non-linearity (confusion) into the cipher, rendering it highly resistant to linear and differential cryptanalysis.

To achieve this, the S-Box is constructed through two mathematical steps. First, each element is mapped to its multiplicative inverse in the finite Galois Field GF(2^8). The multiplicative inverse x^{-1} of a value x in this finite field is defined as the value that, when multiplied by x, yields 1:

$$x \cdot x^{-1} \equiv 1 \pmod{x^8 + x^4 + x^3 + x + 1} \tag{3.1}$$

The irreducible polynomial $x^8 + x^4 + x^3 + x + 1$ corresponds to the binary sequence 100011011_2. This specific polynomial was selected by the AES creators to ensure the output is strictly constrained within an 8-bit length. By definition, the inverse of `0x00` is mapped to itself.

Secondly, to break algebraic symmetries and prevent structural attacks, each multiplicative inverse b undergoes a specific affine transformation over GF(2^8):

$$b' = (A \cdot b) \oplus c$$

where A is a fixed $8 \cdot 8$ bit matrix, and c is the constant column vector $[1,1,0,0,0,1,1,0]^T$ (corresponding to `0x63`).

Vulnerability of the Software LUT Implementation Because dynamically calculating the multiplicative inverse in GF(2^8) during every round would introduce an enormous computational overhead, standard software implementations, such as the `tiny-aes-c` library or those deployed on resource-constrained IoT and edge devices, rely entirely on precalculating these transformations and storing them in a static S-Box LUT.

Consequently, the necessity for these constrained devices to repeatedly access the LUT stored in memory or cache generates a deterministic physical energy leakage during each substitution. This power footprint is directly correlated with the specific byte value being processed, creating a vulnerability that can be readily exploited in Power Analysis Side-Channel Attacks.

Conversely, modern high-performance processors mitigate this vulnerability by utilizing dedicated cryptographic hardware extensions (e.g., Intel AES-NI or ARM CE). These hardware modules execute AES transformations directly within the CPU using dedicated logic gates, thereby eliminating the need for memory-based LUT lookups and effectively neutralizing this specific class of side-channel leakage.

3.1.4 ShiftRows

The SHIFTROWS transformation is a linear operation primarily responsible for introducing horizontal diffusion across the State matrix. It adheres to the following mathematical formulation:

$$s'_{r,c} = s_{r,(c+r) \bmod 4} \quad \text{for } 0 \leq r < 4 \text{ and } 0 \leq c < 4 \tag{3.2}$$

Following the application of SHIFTROWS, the resulting $4 \cdot 4$ matrix is altered such that the first row remains completely untouched. The second, third, and fourth rows undergo a cyclic left shift of one, two, and three byte positions, respectively.

$$S = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \xrightarrow{\text{SHIFTROWS}} S' = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{bmatrix}$$

While mechanically straightforward, this transformation is cryptographically vital. By shifting the rows, it ensures that the four bytes originally comprising a single column are distributed evenly across four distinct columns. Consequently, when the subsequent MIXCOLUMNS transformation is applied, it will mathematically mix bytes from different original columns, guaranteeing a rapid and robust propagation of the avalanche effect throughout the entire State.

3.1.5 MixColumns

Complementing the horizontal diffusion introduced by SHIFTROWS, the MIXCOLUMNS transformation is the primary operation responsible for applying vertical diffusion across the State matrix. This function ensures that each output byte in a column is mathematically dependent on every input byte within that same column.

To achieve this, each column is treated as a four-term polynomial and multiplied by a fixed matrix over the Galois Field GF(2^8):

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \quad \text{for } 0 \leq c < 4 \tag{3.3}$$

By expanding the matrix multiplication, where $\bullet$ denotes multiplication in GF(2^8) and $\oplus$ denotes bitwise XOR (addition in GF(2^8)), the individual bytes of the new column are calculated as follows:

$$\begin{aligned} s'_{0,c} &= (02 \bullet s_{0,c}) \oplus (03 \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c} \\ s'_{1,c} &= s_{0,c} \oplus (02 \bullet s_{1,c}) \oplus (03 \bullet s_{2,c}) \oplus s_{3,c} \\ s'_{2,c} &= s_{0,c} \oplus s_{1,c} \oplus (02 \bullet s_{2,c}) \oplus (03 \bullet s_{3,c}) \\ s'_{3,c} &= (03 \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (02 \bullet s_{3,c}) \end{aligned} \tag{3.4}$$

Software Implementation of GF(2^8) Multiplication: xtime macro From a practical engineering perspective, performing heavy polynomial multiplications dynamically during every round would be computationally prohibitive for embedded systems. However, the matrix coefficients

were deliberately selected by the AES designers to be exceptionally small (`01`, `02`, and `03`) to allow for radical software optimization.

In lightweight C implementations such as `tiny-aes-c`, multiplication by `02` in $\mathrm{GF}(2^8)$ is elegantly implemented using a macro function typically named `xtime`. This operation consists of a simple bitwise left shift of the byte, followed by a conditional XOR with the irreducible polynomial `0x1B` if the most significant bit prior to the shift was 1.

To rigorously verify this optimization, the generic input byte must be represented as a polynomial:

$$A(x) = a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

where the coefficients $a_7, \ldots, a_0$ are individual bits, with a_7 being the Most Significant Bit (MSB).

The hexadecimal value `0x02` corresponds to the binary string `00000010`, which is equivalent to the polynomial x. Therefore, multiplying $A(x)$ by `0x02` yields:

$$A(x) \cdot x = a_7x^8 + a_6x^7 + a_5x^6 + a_4x^5 + a_3x^4 + a_2x^3 + a_1x^2 + a_0x$$

In computer architecture, multiplying by x corresponds exactly to a logical bitwise left shift of one position. However, to ensure the result remains within the finite field $\mathrm{GF}(2^8)$, the maximum degree of the resulting polynomial must not exceed x^7. This constraint presents two possible cases:

- $a_7 = 0$**:** After the multiplication, the coefficient of x^8 is 0. The highest degree remains x^7, and no further mathematical reduction is necessary.
- $a_7 = 1$**:** The resulting polynomial contains an x^8 term, thus falling outside the finite field. To bring the result back into $\mathrm{GF}(2^8)$, it must be divided by the AES irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$, and replaced by the remainder of this division. In binary arithmetic over $\mathrm{GF}(2)$, this modulo reduction corresponds precisely to an XOR operation with the irreducible polynomial:
$$(A(x) \cdot x) \oplus (x^8 + x^4 + x^3 + x + 1)$$

Building on this logic, multiplication by `03` is effortlessly computed. Since `0x03` corresponds to the binary `00000011` (the polynomial $x+1$), the multiplication $A(x) \cdot$ `0x03` expands to:

$$A(x) \cdot (x+1) = (A(x) \cdot x) + (A(x) \cdot 1) = (A(x) \cdot x) + A(x)$$

The first term, $(A(x) \cdot x)$, is exactly the `02` multiplication (`xtime`) just demonstrated. Meanwhile, polynomial addition in $\mathrm{GF}(2^8)$ is mathematically equivalent to a simple bitwise XOR. Thus, the operation seamlessly translates to $(\texttt{02} \bullet A) \oplus A$.

This architectural design guarantees that the MIXCOLUMNS transformation remains exceptionally fast and highly efficient, relying strictly on basic CPU bitwise operations without necessitating additional memory-heavy LUTs.

3.1.6 AddRoundKey

The ADDROUNDKEY transformation is the first operation applied in the algorithm. It consists of a straightforward bitwise XOR operation between the current *State matrix* and the four words of the *expanded key* corresponding to the current round.

For a given column c (where $0 \leq c < 4$) and a specific round, the transformation is defined as:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix} \oplus \begin{bmatrix} w_{4\cdot\text{round}+c} \end{bmatrix}$$

Consider, for instance, the application of the ADDROUNDKEY transformation to the first column of the state ($c = 0$) during the first round. Let the initial state column and the corresponding word from the expanded key array (w_4) be defined as follows:

$$\begin{bmatrix} s_{0,0} \\ s_{1,0} \\ s_{2,0} \\ s_{3,0} \end{bmatrix} = \begin{bmatrix} \texttt{04} \\ \texttt{E0} \\ \texttt{48} \\ \texttt{28} \end{bmatrix} \quad \text{and} \quad w_{4\cdot 1+0} = w_4 = \begin{bmatrix} \texttt{D6} \\ \texttt{AA} \\ \texttt{74} \\ \texttt{FD} \end{bmatrix}$$

By applying the bitwise XOR ($\oplus$) between the two column vectors, the resulting updated state column is computed as:

$$\begin{bmatrix} s'_{0,0} \\ s'_{1,0} \\ s'_{2,0} \\ s'_{3,0} \end{bmatrix} = \begin{bmatrix} \texttt{04} \\ \texttt{E0} \\ \texttt{48} \\ \texttt{28} \end{bmatrix} \oplus \begin{bmatrix} \texttt{D6} \\ \texttt{AA} \\ \texttt{74} \\ \texttt{FD} \end{bmatrix} = \begin{bmatrix} \texttt{D2} \\ \texttt{4A} \\ \texttt{3C} \\ \texttt{D5} \end{bmatrix} \tag{3.5}$$

3.2 Galois/Counter Mode (GCM)

To apply a fixed-size block cipher such as AES to variable-length inputs, it is necessary to employ a specific *mode of operation*. A mode of operation dictates the exact architectural procedure by which the core encryption algorithm processes data streams larger than its native 128-bit block size.

For instance, the most rudimentary mode of operation, *Electronic Codebook (ECB)*, starts by evaluating whether the plaintext length is an exact multiple of 128 bits. If it is not, a padding scheme must be appended to the end of the message. The plaintext is then partitioned into 128-bit blocks, and each block is independently fed into the AES encryption algorithm. Finally, the resulting encrypted blocks are sequentially concatenated to form the complete ciphertext.

However, the ECB mode is notoriously vulnerable to cryptanalysis. Because the encryption of each block is entirely independent and deterministic, two identical plaintext blocks encrypted under the same key will invariably produce the exact same ciphertext blocks. This fatal flaw fails

to eliminate large-scale redundancies, preserving the structural patterns of the original data and rendering it cryptographically inadequate for secure communications.

To address these deterministic flaws, more advanced modes of operation were developed, such as the *Counter Mode (CTR)*. Unlike ECB, the CTR mode does not feed the plaintext directly into the cipher. Instead, it utilizes a 128-bit block comprising a unique *nonce* (Number Used Once) concatenated with an internal counter.

In this architecture, the input to the AES algorithm is the counter block itself. The output of the AES encryption (a pseudo-random key stream) is then directly XORed with the current block of the plaintext to produce the ciphertext. For each subsequent block of the message, the value of the counter is incremented by one, encrypted by the block cipher, and XORed with the next plaintext block. This operation is repeated until the entire message is processed.

The utilization of the counter as the AES input, instead of the plaintext block itself, guarantees the CTR mode a complete lack of inter-block dependency, representing a major architectural advantage.

By leveraging the possibility of pre-calculating the sequence of values that the counter will assume, the AES encryption operations can be highly parallelized, thereby achieving significantly higher throughput compared to sequential modes.

Although the baseline `tiny-aes-c` library [2] does not natively provide a GCM implementation, utilizing this specific mode is mandatory to accurately replicate the cryptographic environment characteristic of modern SATCOM systems. Consequently, as a core experimental contribution of this study, the GCM architecture was custom-implemented on top of the existing block cipher. This development was executed by strictly adhering to the official guidelines and mathematical specifications defined in NIST SP 800-38D [14].

The GCM Extension: Authenticated Encryption and Integrity Building upon the architecture of the CTR mode, the *Galois/Counter Mode (GCM)* expands the cryptographic framework by incorporating a robust authentication layer to ensure the integrity of the transmitted message. To achieve this, GCM implements an *Authenticated Encryption with Associated Data (AEAD)* scheme.

In a typical network transmission, certain information must remain in plaintext to allow routers and gateways to correctly handle the packets. This is defined as the *Additional Authenticated Data (AAD)*. The AAD typically comprises essential metadata such as network addresses, protocol versions, routing ports, and sequence numbers. The inclusion of sequence numbers is particularly critical in SATCOM environments to prevent *replay attacks*, a severe threat involving the malicious interception and subsequent retransmission of a previously valid message.

While the AAD is transmitted unencrypted, its content, alongside the encrypted ciphertext, is mathematically processed by the algorithm to generate a unique 128-bit *authentication tag*.

During transmission, this authentication tag is appended to the message payload (which consists of the AAD and the ciphertext). Upon reception, the receiver replicates the exact same cryptographic process using the shared secret key, the received AAD, and the ciphertext to produce a new, localized authentication tag. If this locally generated tag perfectly matches the received tag,

the receiver can safely conclude that neither the ciphertext nor the plaintext metadata was altered, corrupted, or tampered with during transmission.

3.2.1 Key and IV Requirements

While the standard CTR mode utilizes a unique nonce as the prefix for its counter, the GCM architecture delegates this critical role to the *Initialization Vector (IV)*. To guarantee the designed cryptographic security bounds, strict regulatory requirements regarding the selection and management of both the key and the IV must be satisfied.

Uniqueness and Forgery Attacks. The most critical constraint of AES-GCM is the *uniqueness requirement*. The standard dictates that the probability of the authenticated encryption function being invoked with the same Key/IV pair on two distinct plaintexts must be strictly less than 2^{-32}.

The consequences of violating this requirement by reusing an IV with the same key are catastrophic. By construction, the GHASH authentication tag is derived via polynomial evaluations over GF(2^8) involving the ciphertext and the IV. If an IV is repeated, an attacker can mathematically recover the internal authentication subkey. This exposes the system to *forgery attacks*, allowing the attacker to produce fake messages with valid authentication tags, completely compromising the integrity of the channel. In practice, safeguarding IV uniqueness is considered as vital as maintaining the secrecy of the master key itself.

Key Freshness and Establishment. In addition to the IV, key management plays a pivotal role. The principles of *key uniqueness* and *key establishment* mandate that for each new communication session, the generated symmetric key must be "fresh" (cryptographically unpredictable). Furthermore, the communication channel utilized to distribute this master key (often asymmetric or out-of-band) must be highly secure to prevent key theft prior to the AES-GCM execution.

IV Construction Methods. Because of the vital role the IV plays, NIST SP 800-38D[14] outlines strict construction rules, heavily recommending a standard IV length of 96 bits for optimal performance and security. Depending on the system architecture, two main generation processes are defined:

- **Deterministic Construction:** This method is mandatory for IVs strictly shorter than 96 bits, but is widely used for 96-bit IVs as well. It stipulates that the IV is divided into two sections: a *fixed field* and an *invocation field*. The fixed field uniquely identifies the device or the specific instance of the encryption algorithm (e.g., a satellite ID), whereas the invocation field acts as a sequential counter for the data sets. To maintain uniqueness, two devices sharing the same key must never share the same fixed field, and the invocation field must increment deterministicly.
- **RBG-based Construction:** For IVs of 96 bits or longer, a *Random Bit Generator (RBG)* construction can be employed. This format consists of a *random field* and an optional *free*

field. To satisfy the 2^{-32} collision probability threshold, the random field must have a minimum length of 96 bits and be generated by a cryptographically strong RBG. The free field has no structural requirements and can contain arbitrary device metadata, though the standard suggests leaving it empty to maximize the entropy of a fully randomized IV.

3.2.2 Multiplication in GF(2^{128})

As previously introduced in the MIXCOLUMNS analysis, calculations within Galois Finite Fields are utilized to guarantee mathematical closure and strong diffusion, ensuring that outputs remain strictly within a defined range while maintaining the influence of every input bit. Correspondingly, one of the primary mathematical operations underlying the GCM authenticated encryption scheme is Galois multiplication applied over the extended finite field GF(2^{128}).

Within GF(2^{128}), 128-bit blocks are treated as polynomials with a maximum degree of 127. Each bit of the data block represents the coefficient of the corresponding polynomial term, ranging from x^0 to x^{127}.

Algorithm 2 Multiplication in GF(2^{128}): $X \bullet Y$

```
1:  procedure GALOISMULTIPLICATION(X, Y)
2:      R ← 11100001_2 || 0^120
3:      Z ← 0^128
4:      V ← Y
5:      for i ← 0 to 127 do
6:          if x_i = 1 then                          ▷ x_i is the i-th bit of block X
7:              Z ← Z ⊕ V
8:          end if
9:          if LSB_1(V) = 0 then
10:             V ← (V ≫ 1)
11:         else
12:             V ← (V ≫ 1) ⊕ R
13:         end if
14:     end for
15:     return Z
16: end procedure
```

The algorithmic translation of this multiplication logic begins with the initialization of three primary variables `(lines 2-4)`:

- R: Represents the reduction polynomial block, which is used to bring the result back into the finite field whenever a polynomial multiplication exceeds the maximum degree constraint. The formal irreducible polynomial chosen by the GCM designers is $f(x) = x^{128} + x^7 + x^2 + x + 1$. During modulo reduction in GF(2^{128}), the x^{128} term is mathematically substituted by $x^7 + x^2 + x + 1$. Following the NIST SP 800-38D standard [14], which adopts a adopts a *bit-reflected* convention, operating analogously to a little-endian scheme, where the leftmost bit is interpreted as the x^0 coefficient, this reduction polynomial is mapped to the binary string $11100001_2 \,||\, 0^{120}$ (equivalent to the hexadecimal block `0xE100...00`).
- Z: The accumulator variable, initialized as an array of 128 zeros (0^{128}).

- V: The multiplicand variable, initialized as a direct copy of the input block Y.

To calculate the product $X \bullet Y$ in GF(2^{128}), the algorithm employs a process known as *Shift-and-Add*, which mathematically mirrors classical column-wise multiplication.

Considering that the 128-bit block X is treated as a polynomial with a maximum degree of 127:

$$X = x_0 + x_1 x + x_2 x^2 + \cdots + x_{127} x^{127}$$

by applying the distributive property, the multiplication between X and Y can be expanded as follows (where modulo 2 addition is logically equivalent to the bitwise XOR operator $\oplus$):

$$X \bullet Y = (x_0 \cdot Y) \oplus (x_1 \cdot Y \cdot x) \oplus (x_2 \cdot Y \cdot x^2) \oplus \cdots \oplus (x_{127} \cdot Y \cdot x^{127})$$

Within the algorithm's pseudocode, this mathematical expansion is implemented inside the main loop (`lines 5-14`). The execution of each iteration can be broken down into two distinct operational phases: the accumulation phase and the shift-and-reduce phase.

Phase 1: Accumulation (Lines 6–8) During the i-th iteration, the algorithm inspects the state of x_i (the i-th bit of X) to determine its contribution to the final product:

- **If $x_i = 1$:** The mathematical term evaluates to $Y \cdot x^i$. Consequently, the current value of the multiplicand V (which currently holds the value $Y \cdot x^i$) is XORed into the accumulator Z (`line 7`).
- **If $x_i = 0$:** The term evaluates to 0. Since zero is the identity element for the XOR operation, it acts as a null term, and the accumulator Z remains unchanged.

Phase 2: Shift and Reduce (Lines 9–13) In the final phase of the iteration, the state of the multiplicand V must be updated for the next loop by multiplying it by x (effectively elevating its polynomial degree). Crucially, because the NIST SP 800-38D standard mandates a *bit-reflected* (little-endian) convention, elevating the polynomial degree corresponds to a **logical right shift** ($\gg 1$) at the architectural level, rather than a traditional left shift.

Moreover, to guarantee that the temporary result remains strictly within the GF(2^{128}) boundaries, a boundary check is executed on the highest degree coefficient prior to the shift (`line 9`). Due to the bit-reflected nature of the data, the highest degree coefficient (x^{127}) is physically located at the Least Significant Bit (LSB_1) of the array. If this bit is 0, a simple right shift is sufficient. However, if the LSB is 1, the subsequent right shift would cause an overflow outside the finite field. To mathematically correct this, the shifted result is immediately XORed with the irreducible polynomial R (`line 12`), seamlessly reducing the value back into the valid finite field space.

3.2.3 GHASH Function

The generation of the authentication tag is the theoretical core of the GCM architecture, and the GHASH function serves as the mathematical engine utilized by the main encryption algorithm to achieve this integrity guarantee.

Despite its name, GHASH is not a traditional unkeyed cryptographic hash function (such as SHA-256), but rather a *universal hash function*. It is specifically designed to compute a 128-bit message authentication code (MAC) by evaluating a polynomial over $\mathrm{GF}(2^{128})$. When combined with the strict IV uniqueness requirements, it provides an extremely low probability of differential output collisions.

The cornerstone of the GHASH design is the utilization of the *hash subkey* H. This subkey is derived by applying the block cipher to a 128-bit block of zeros under the master key, meaning $H = \mathrm{AES}_K(0^{128})$. Ultimately, the core operation executed within this function is the GALOIS-MULTIPLICATION previously defined, which processes the data blocks iteratively.

Algorithm 3 GHASH_H Function

1: **procedure** $\mathrm{GHASH}_H(X)$
2: $m \leftarrow \mathrm{len}(X)/128$
3: Let $X_1, X_2, \ldots, X_m$ denote the blocks such that $X = X_1 \| X_2 \| \ldots \| X_m$
4: $Y \leftarrow 0^{128}$
5: **for** $i \leftarrow 1$ **to** m **do**
6: $Y \leftarrow \mathrm{GALOISMULTIPLICATION}(Y \oplus X_i, H)$ ▷ Multiplication in $\mathrm{GF}(2^{128})$
7: **end for**
8: **return** Y
9: **end procedure**

As detailed in Algorithm 3, the variable m denotes the total number of 128-bit blocks comprising the input X (`line 2`). The sequence $X_1, X_2, \ldots, X_m$ represents each sequential 128-bit section of the input (`line 3`), followed by the initialization of Y, a zero-block utilized as the accumulator variable (`line 4`). Finally, for each i-th block X_i, the accumulator Y is mathematically updated with the output of the Galois multiplication between the hash subkey H and the XORed state of $Y \oplus X_i$.

Embedded C Implementation and Memory Optimization. In space-grade embedded systems, dynamically allocating memory (e.g., utilizing the heap via `malloc`) for variable-length inputs is strictly avoided to prevent memory fragmentation and ensure deterministic execution. Conversely, copying massive payloads directly onto the stack would immediately lead to stack overflow faults.

To address these severe architectural constraints within the `tiny-aes-c` extension [2], the theoretical GHASH algorithm was not implemented as a standalone function. Instead, the mathematical operations are executed *in-place* inside the main authenticated encryption and decryption routines. As shown in the code snippet below, the implementation iteratively processes the Additional Authenticated Data (AAD) and the ciphertext in 16-byte chunks, XORing them directly into the accumulator `S` and immediately calling the `galoisMultiplication` function.

```
// Processing AAD in blocks of 16 bytes without dynamic memory allocation
for (size_t i = 0; i < aad_length; i += AES_BLOCKLEN) {
    for (size_t j = 0; j < AES_BLOCKLEN; j++) {
        if (i + j < aad_length) {
            S[j] ^= aad[i + j];
        }
    }
    galoisMultiplication(S, H, S);
}
```

This engineering approach entirely circumvents the need to duplicate large data structures in memory, rigorously adapting the high-level NIST specifications to the harsh constraints of lightweight satellite microcontrollers.

3.2.4 Authenticated Encryption and Decryption Algorithms

Ultimately, all the cryptographic primitives and modes of operation detailed in this chapter converge into the final Authenticated Encryption with Associated Data (AEAD) algorithms. These procedures orchestrate the encryption process to produce a secure ciphertext accompanied by an authentication tag, and conversely, manage the decryption process while cryptographically verifying the message's authenticity through tag comparison.

Authenticated Encryption Algorithm The encryption algorithm operates on the following set of input variables:

- K: The secret 256-bit *master key*.
- IV: The variable-length *Initialization Vector*, chosen in accordance with official NIST specifications [14, p. 18–21]. (In modern SATCOM standards, this is conventionally fixed at 96 bits / 12 bytes).
- P: The variable-length input *plaintext*.
- A: The *Additional Authenticated Data (AAD)*.
- t: An integer value determining the maximum bit-length of the generated authentication tag.

Moreover, a set of auxiliary subroutines is utilized during execution. The $\textsc{Ciph}_K()$ routine invokes the core AES block cipher to encrypt a 128-bit zero-block, producing the hash subkey H. The $\text{inc}_{32}()$ function takes a 128-bit block and increments its 32 rightmost bits, which act as a counter that wraps around to zero upon overflow. Lastly, $\textsc{Gctr}_K()$ executes the standard CTR mode of operation for a fixed key K, processing a given input payload starting from a specified initial counter block.

The procedure initiates by computing the hash subkey H via the $\textsc{Ciph}_K()$ routine `(line 2)`. Subsequently, a conditional structure determines the format of the initial counter block J_0 `(lines 3-8)`. If the IV is exactly 96 bits long, J_0 is constructed by concatenating the IV with a 32-bit

Algorithm 4 GCM Authenticated Encryption

1: **procedure** GCMAUTHENTICATEDENCRYPT(K, IV, P, A, t)
2: $H \leftarrow \text{CIPH}_K(0^{128})$ ▷ Generate the hash subkey H
3: **if** $\text{len}(IV) = 96$ **then**
4: $J_0 \leftarrow IV \,\|\, 0^{31} \,\|\, 1$
5: **else**
6: $s \leftarrow 128 \cdot \lceil \text{len}(IV)/128 \rceil - \text{len}(IV)$
7: $J_0 \leftarrow \text{GHASH}_H(IV \,\|\, 0^{s+64} \,\|\, [\text{len}(IV)]_{64})$
8: **end if**
9: $C \leftarrow \text{GCTR}_K(\text{inc}_{32}(J_0), P)$ ▷ Encrypt plaintext P
10: $u \leftarrow 128 \cdot \lceil \text{len}(C)/128 \rceil - \text{len}(C)$
11: $v \leftarrow 128 \cdot \lceil \text{len}(A)/128 \rceil - \text{len}(A)$
12: $S \leftarrow \text{GHASH}_H(A \,\|\, 0^v \,\|\, C \,\|\, 0^u \,\|\, [\text{len}(A)]_{64} \,\|\, [\text{len}(C)]_{64})$
13: $T \leftarrow \text{MSB}_t(\text{GCTR}_K(J_0, S))$ ▷ Compute authentication tag of length t
14: **return** (C, T)
15: **end procedure**

counter initialized to 1. Conversely, if the IV length differs from 96 bits, an integer s is calculated to determine the required zero-padding. The counter block J_0 is then generated by passing the IV, the zero-padding, and a 64-bit representation of the IV's length through the $\text{GHASH}_H()$ function.

Following initialization, the ciphertext C is produced using the $\text{GCTR}_K()$ routine, utilizing the incremented initial counter (which resolves to `02` for a 96-bit IV) and the plaintext P as inputs `(line 9)`.

Continuing the process, two integer values, u and v, are calculated to determine the required zero-padding based on the lengths of the ciphertext and the AAD, respectively `(lines 10-11)`. An intermediate accumulator block S is then produced via the $\text{GHASH}_H()$ operation by evaluating the concatenation of the AAD, its padding, the ciphertext, its padding, and the 64-bit length representations of both the AAD and the ciphertext `(line 12)`. Ultimately, the authentication tag T is generated by encrypting S with the original initial counter J_0, extracting only the t most significant bits `(line 13)`.

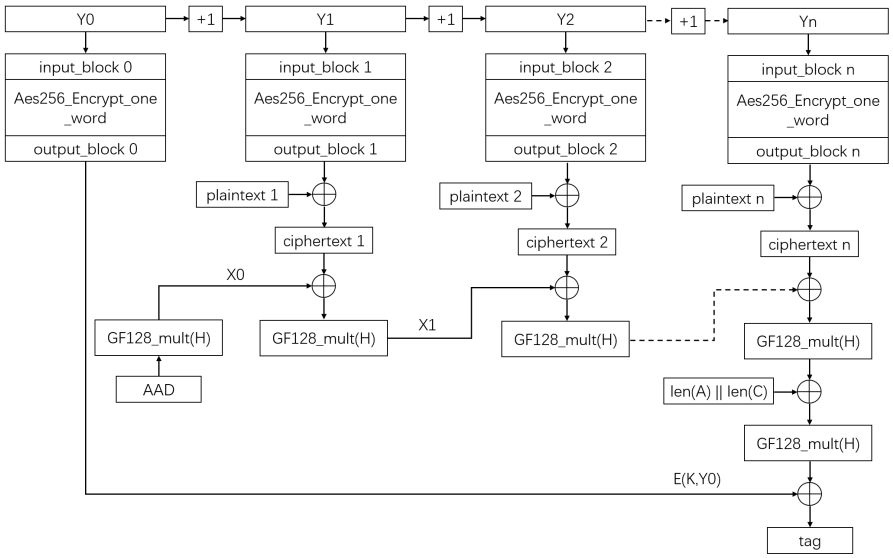


Figure 3.3: AES-GCM Authenticated Encryption architecture detailing the data flow and tag generation process [19].

Authenticated Decryption Algorithm The authenticated decryption algorithm shares the vast majority of its cryptographic logic with its encryption counterpart. The primary operational differences reside in the initial validation checks—which verify that the bit-lengths of the IV, AAD, and ciphertext are supported, and that the provided tag matches the expected length t. In addition, also the transposition of the input variable C (ciphertext) in place of P (plaintext) is a main differences.

Crucially, after following the standard computational flow to produce a localized tag T', the algorithm executes a constant-time comparison between T' and the received tag T. If the two tags match perfectly, the authentication succeeds, and the decrypted plaintext is released. Otherwise, the message is deemed compromised, either through malicious tampering or transmission errors. In this failure state, the algorithm immediately aborts and returns a FAIL error `(lines 17-18)`, strictly discarding the plaintext to prevent Chosen-Ciphertext Attacks (CCA).

Algorithm 5 GCM Authenticated Decryption

1: **procedure** GCMAUTHENTICATEDDECRYPT(K, IV, C, A, T, t)
2: **if** bit lengths of IV, A, or C are unsupported **or** $\text{len}(T) \neq t$ **then**
3: **return** FAIL
4: **end if**
5: $H \leftarrow \text{CIPH}_K(0^{128})$ ▷ Generate the hash subkey H
6: **if** $\text{len}(IV) = 96$ **then**
7: $J_0 \leftarrow IV \,\|\, 0^{31} \,\|\, 1$
8: **else**
9: $s \leftarrow 128 \cdot \lceil \text{len}(IV)/128 \rceil - \text{len}(IV)$
10: $J_0 \leftarrow \text{GHASH}_H(IV \,\|\, 0^{s+64} \,\|\, [\text{len}(IV)]_{64})$
11: **end if**
12: $P \leftarrow \text{GCTR}_K(\text{inc}_{32}(J_0), C)$ ▷ Decrypt ciphertext C
13: $u \leftarrow 128 \cdot \lceil \text{len}(C)/128 \rceil - \text{len}(C)$
14: $v \leftarrow 128 \cdot \lceil \text{len}(A)/128 \rceil - \text{len}(A)$
15: $S \leftarrow \text{GHASH}_H(A \,\|\, 0^v \,\|\, C \,\|\, 0^u \,\|\, [\text{len}(A)]_{64} \,\|\, [\text{len}(C)]_{64})$
16: $T' \leftarrow \text{MSB}_t(\text{GCTR}_K(J_0, S))$ ▷ Compute expected authentication tag
17: **if** $T = T'$ **then**
18: **return** P ▷ Authentication succeeded
19: **else**
20: **return** FAIL ▷ Integrity check failed
21: **end if**
22: **end procedure**

3.2.5 Tests for the GCM Embedded C Implementation

NIST Vector Tests To rigorously verify the cryptographic correctness of the custom C implementation, a selected suite of test vectors was utilized [20]. These test vectors are provided by the *Cryptographic Algorithm Validation Program (CAVP)*, established directly by NIST.

The suite contains tens of thousands of individual test cases for both the authenticated encryption and decryption processes. Each test evaluates the algorithm against variable-length plaintexts, ciphertexts, IVs, and AADs, utilizing diverse 256-bit master keys and their corresponding expected authentication tags.

The execution of these tests yielded a 100% success rate, demonstrating that the embedded implementation is mathematically sound and fully compliant with the FIPS and NIST standards.

Channel Noise and SEU Simulation Beyond formal cryptographic validation, a practical simulation was developed to replicate the effects of *Additive White Gaussian Noise (AWGN)* and radiation-induced *Single Event Upsets (SEUs)*, which represent critical concerns in SATCOM environments.

It is vital to verify that the AEAD architecture correctly identifies arbitrary bit-level message alterations caused by the harsh space environment. To emulate a transmission error over a noisy channel, a random bit-flip was artificially injected into a valid ciphertext prior to the decryption phase.

```
Plaintext (429 byte): SAT-7 telemetry window 2041: the spacecraft is in nominal sun-pointing
    attitude and the Ka-band link remains locked with the ground station in Matera. The flight
    computer reports stable battery voltage, adequate thermal margins, and no corrected memory
    errors during the last pass. Payload data will be downlinked in two scheduled contact
    windows; the receiver shall request a repeat only if the authenticated packet check fails.
AAD (95 byte): SAT-7|pass=2041|ground-station=Matera|band=Ka|channel=telemetry|priority=nominal|
    counter=000184
Randomly flipped bit 1896 of the ciphertext.
Received Tag: 518735e4ea1697d3dcf9f0bb69ae65e4
ERROR: decryption failed as expected due to the bit flip.
```

With the baseline AES-256-GCM architecture fully implemented, theoretically analyzed, and functionally validated, the foundational framework of this study is established. The next chapter will address the physical vulnerabilities of this standard implementation by integrating a chaotic injection mechanism into the S-Box. Drawing inspiration from the foundational concepts proposed by Kumar et al. [1], the following chapter explores a cryptographic "Machine Unlearning" approach, deliberately adapting it to break the deterministic power leakages exploited by advanced ML-based Side-Channel Attacks in SATCOM environments.

Chapter 4

Chaotic Injection and Cryptographic Machine Unlearning

Bibliography

[1] Amit Krishan Kumar et al. “Advanced Encryption Techniques in Satellite Communications With Chaotic Injection and Quantum Collapse for Enhanced Security”. In: *International Journal of Satellite Communications and Networking* 44.2 (2026), pp. 158–170. DOI: https://doi.org/10.1002/sat.70009. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/sat.70009. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/sat.70009.

[2] Kasper Kokke. *tiny-AES-c: Small Portable AES128/192/256 in C*. 2020. URL: https://github.com/kokke/tiny-AES-c.

[3] Consultative Committee for Space Data Systems (CCSDS). *Space Data Link Security Protocol*. Recommendation for Space Data System Standards CCSDS 355.0-B-2. Washington, D.C., USA, July 2022. URL: https://ccsds.org/Pubs/355x0b2.pdf.

[4] Sara Salim, Nour Moustafa, and Martin Reisslein. “Cybersecurity of Satellite Communications Systems: A Comprehensive Survey of the Space, Ground, and Links Segments”. In: *IEEE Communications Surveys & Tutorials* 27.1 (2025), pp. 372–425. DOI: 10.1109/COMST.2024.3408277.

[5] Weidong Zhang et al. “Satellite as a Service: a Hybrid Resource Management Framework for Space-terrestrial Integrated Networks”. In: *2020 IEEE 11th International Conference on Software Engineering and Service Science (ICSESS)*. 2020, pp. 171–174. DOI: 10.1109/ICSESS49938.2020.9237705.

[6] Wikimedia Commons. *Diagram of the Space, Ground, and User Segments in a Satellite Communications System*. Accessed: 06-11-2026. 2026. URL: https://commons.wikimedia.org/wiki/File:Space_Ground_User_segments.png.

[7] Aditi Ranjit Kumar Verma. “Cybersecurity in Satellite Communication Networks: Key Threats and Neutralization Measures”. In: *IEEE Open Journal of the Communications Society* 6 (2025), pp. 5667–5692. DOI: 10.1109/OJCOMS.2025.3585060.

[8] Nora Abdelsalam, Saif M. Al-Kuwari, and Aiman M. Erbad. “Physical layer security in satellite communication: State-of-the-art and open problems”. In: *IET Communications* 19.1 (2025), e12830. DOI: 10.1049/cmu2.12830. URL: https://doi.org/10.1049/cmu2.12830.

[9] Suhyeon Jeon, Jeongho Kwak, and Jihwan P. Choi. “Cross-Layer Encryption of CFB-AES-TURBO for Advanced Satellite Data Transmission Security”. In: *IEEE Transactions on Aerospace and Electronic Systems* 58.3 (2022), pp. 2192–2205. DOI: 10.1109/TAES.2021.3134988.

[10] *Space Data Link Security Protocol—Extended Procedures*. Recommended Standard CCSDS 355.1-B-1. Washington, D.C., Feb. 2020. URL: https://ccsds.org/Pubs/355x1b1.pdf.

[11] Mahdi Baghaei Jezehei, Seyed Ahmad Olamaei, and Ali Broumandnia. "Lightweight secure message transfer protocol based on ECC (Nist P-256) Internet of Things equipped with satellite communications". In: *International Journal of Nonlinear Analysis and Applications* 16.4 (2025), pp. 93–102. ISSN: 2008-6822. DOI: 10.22075/ijnaa.2023.32465.4831. eprint: https://ijnaa.semnan.ac.ir/article_8995_d469547325a320e660ba7f4bf05c7ecf.pdf. URL: https://ijnaa.semnan.ac.ir/article_8995.html.

[12] Neeraj Sharma. "Investigating the Impact of Quantum Computing on Existing Encryption Methods". In: *Journal of Information Systems Engineering and Management* 10 (May 2025), pp. 994–1010. DOI: 10.52783/jisem.v10i41s.8035.

[13] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication NIST FIPS 197-upd1. National Institute of Standards and Technology, May 2023. DOI: 10.6028/NIST.FIPS.197-upd1. URL: https://doi.org/10.6028/NIST.FIPS.197-upd1.

[14] Morris Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. Gaithersburg, MD: National Institute of Standards and Technology (NIST), Nov. 2007. DOI: 10.6028/NIST.SP.800-38D. URL: https://csrc.nist.gov/pubs/sp/800/38/d/final.

[15] Hayder Yasir Naser et al. "A Comparison of Lightweight Cryptographic Protocols for Energy-Efficient and Sustainable IoMT Authentication". In: *Engineering, Technology & Applied Science Research* 15.4 (Aug. 2025), pp. 25746–25756. DOI: 10.48084/etasr.12204. URL: https://etasr.com/index.php/ETASR/article/view/12204.

[16] Brandon Tsao, Yuhong Liu, and Behnam Dezfouli. "Analysis of the duration and energy consumption of AES algorithms on a contiki-based IoT device". In: Nov. 2019, pp. 483–491. DOI: 10.1145/3360774.3368202.

[17] Durga Prasad Chikkala. *AES (Advanced Encryption Standard)*. 2024. URL: https://medium.com/@durgachikkaladev/aes-advanced-encryption-standard-88e440cf0525 (visited on 11/08/2026).

[18] Muhammad Asif Bin Siddique et al. "An Efficient Hardware Implementation of AES S-Box". In: (2019). Figure 3: S-box Values for All 256 Combinations in Hexadecimal Format. URL: https://www.researchgate.net/figure/S-box-Values-for-All-256-Combinations-in-Hexadecimal-Format-1_fig3_335740212 (visited on 11/08/2026).

[19] Xilinx, Inc. *Vitis Security Library: Galois/Counter Mode (GCM)*. Versione 2019.2. 2019. URL: https://xilinx.github.io/Vitis_Libraries/security/2019.2/guide_L1/internals/gcm.html (visited on 11/10/2026).

[20] National Institute of Standards and Technology. *Cryptographic Algorithm Validation Program (CAVP): Block Cipher Modes Validation System (GCMVS) Test Vectors*. NIST Computer Security Resource Center. 2020. URL: https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program/cavp-testing-block-cipher-modes (visited on 11/11/2026).